



兰州工业学院

实 验 报 告

(2023 —2024 学年第 1 学期)

课程名称 面向对象程序设计

专业班级 专升本电信 23

学 号 ZB2305010116

姓 名 姜 勇

实验项目	实验七 文件的存储和读取		
实验时间	2023 年 12 月 13 日	实验地点	信息与通信工程专业实验室 (DSP)
一、实验目的 1. 掌握文件的存储和读取; 2. 理解使用文件、文本流、数据流进行文件操作的不同。 二、实验环境 硬件：计算机一台; 软件：VC++ 6.0 或 Code::Blocks、Qt 开发系统。 三、实验内容 实现一个能够读取文件内容和存储文件内容的窗口程序。具体要求如下： (1) 主窗口标题名称为学号后两位+姓名。 (2) 设置菜单栏，包含“文件操作”、“数据操作”和“帮助”三个菜单。 (3) “文件操作”选择后，可以跳转到文件读取和存储界面，左边是读取页面，右边是存储页面，分别包含了标题、序号、内容页面和操作按钮。点击对应的按钮可以完成文件内容的读取和存储。			
(4) 帮助菜单中，提供软件功能说明和开发者信息说明。 (5) 自由发挥：添加工具栏。			



(6) 自由发挥：完成文件的连续读取和连续写入。

mainwindow.h 文件

```
#ifndef MAINWINDOW_H
#define MAINWINDOW_H
#include <QMainWindow>
class MainWindow : public QMainWindow
{
    Q_OBJECT
public:
    MainWindow(QWidget *parent = nullptr);
    ~MainWindow();
    void showfile();
    void fun();
    void aboutn();
    void writeFiles();
    void rename();
    void copy();
};
#endif // MAINWINDOW_H
```

mainwindow.cpp 文件

```
#include "mainwindow.h"
#include "file.h"
#include <QLabel>
#include <QMenuBar>
#include <QMessageBox>
#include <QToolBar>
#include <QStatusBar>
#include <QFileDialog>
```

```

#include <QInputDialog>
#include <QTextEdit>
MainWindow::MainWindow(QWidget *parent)
    : QMainWindow(parent)
{
    setWindowTitle("16 姜勇");
    resize(500, 500);
    QMenuBar *menuBar=new QMenuBar(this);
    this->setMenuBar(menuBar);
    QMenu *file=new QMenu("文件操作");
    menuBar->addMenu(file);
    QAction *readFileAction = new QAction(QIcon(":/img/open.png"), "&
读取", this);
    connect(readFileAction,          &QAction::triggered,          this,
&MainWindow::showfile);
    file->addAction(readFileAction);
    QMenu *data=new QMenu("数据操作");
    menuBar->addMenu(data);
    QAction *rename=data->addAction(QIcon(":/img/rename.jpg"), "重命
名文件");
    connect(rename, &QAction::triggered, this, &MainWindow::rename);
    QAction *copy=data->addAction(QIcon(":/img/copy.png"), "复制文件
");
    connect(copy, &QAction::triggered, this, &MainWindow::copy);
    QMenu *help=new QMenu("帮助");
    menuBar->addMenu(help);
    QAction *function =help->addAction(QIcon(":/img/new.png"), "软件
功能说明");
    connect(function, &QAction::triggered, this, &MainWindow::fun);

```

```

        QAction *about =help->addAction(QIcon(":/img/exit.png"), "开发者
信息");

        connect (about, &QAction::triggered, this, &MainWindow::aboutn);

        QToolBar *t=new QToolBar(this);

        this->addToolBar(t);

        t->addAction(readFileAction);

        t->addAction(rename);

        t->addAction(copy);

        t->addAction(function);

        t->addAction(about);

        QStatusBar *bar=new QStatusBar();

        this->setStatusBar(bar);

        QLabel *l1=new QLabel(this);

        l1->setText("欢迎使用");

        bar->addWidget(l1);

        QLabel *l2=new QLabel(this);

        l2->setText("创建者: 姜勇");

        bar->addPermanentWidget(l2);

        readFileAction->setStatusTip("读取和存储文件");

        function->setStatusTip("软件说明");

        about->setStatusTip("开发者信息");

    }

void MainWindow::showfile()

{

    file f;

    f.exec();

}

void MainWindow::fun()

{

```

```

        QMessageBox::information(this, "功能说明", "文件的读取和存储");
    }

void MainWindow::aboutn()
{
    QMessageBox::information(this, "开发者信息", "作者：JY\n 时间：2023/12/12");
}

void MainWindow::rename()
{
    // 通过对话框获取用户选择的旧文件路径

    QString oldFilePath = QFileDialog::getOpenFileName(this, tr("选择文件"), ".", tr("Text Files (*.txt)"));

    // 如果用户选择了旧文件路径
    if (!oldFilePath.isEmpty()) {
        // 通过对话框获取用户选择的新文件路径

        QString newFilePath = QFileDialog::getSaveFileName(this, tr("重命名文件"), ".", tr("Text Files (*.txt)"));

        // 如果用户选择了新文件路径
        if (!newFilePath.isEmpty()) {
            // 创建 QFile 对象并指定旧文件路径
            QFile file(oldFilePath);

            // 使用 QFile 的 rename 方法进行重命名操作
            if (file.rename(newFilePath)) {
                QMessageBox::information(this, tr("成功"), tr("文件已成功重命名！"));
            } else {
                QMessageBox::warning(this, tr("失败"), tr("文件重命名失败！"));
            }
        }
    }
}

```

```

        }

    }
}

void MainWindow::copy()
{
    // 通过对话框获取用户选择的源文件路径
    QString sourceFilePath = QFileDialog::getOpenFileName(this, tr("
选择要复制的文件"), ".", tr("All Files (*)"));

    // 如果用户选择了源文件路径
    if (!sourceFilePath.isEmpty()) {
        // 通过对话框获取用户选择的目标文件路径
        QString destinationFilePath =
QFileDialog::getSaveFileName(this, tr("选择复制后的文件路径"), ".",
tr("All Files (*)"));

        // 如果用户选择了目标文件路径
        if (!destinationFilePath.isEmpty()) {
            // 创建 QFile 对象并指定源文件路径
            QFile sourceFile(sourceFilePath);

            // 使用 QFile 的 copy 方法进行文件复制操作
            if (sourceFile.copy(destinationFilePath)) {
                QMessageBox::information(this, tr("成功"), tr("文件已
成功复制!"));
            } else {
                QMessageBox::warning(this, tr("失败"), tr("文件复制失
败!"));
            }
        }
    }
}

```

file.h 文件

```
#ifndef FILE_H
#define FILE_H
#include <QDialog>
namespace Ui {
class file;
}
class file : public QDialog
{
    Q_OBJECT
public:
    explicit file(QWidget *parent = nullptr);
    ~file();
private slots:
    void on_pushButton1_clicked();
    void on_pushButton2_clicked();
    void on_pushButton3_clicked();
    void on_pushButton4_clicked();
private:
    Ui::file *ui;
};
#endif // FILE_H
```

file.cpp 文件

```
#include "file.h"
#include "ui_file.h"
#include <QFile>
#include <QFileDialog>
#include <QElapsedTimer>
#include <QMessageBox>
```



```

file::file(QWidget *parent) :
    QDialog(parent),
    ui(new Ui::file)
{
    ui->setupUi(this);
}

file::~file()
{
    delete ui;
}

void file::on_pushButton1_clicked()
{
    // 通过对话框获取用户选择的源文件路径
    QString filePath = QFileDialog::getOpenFileName(this, tr("选择文件"), ".", tr("Text Files (*.txt)"));
    // 如果用户选择了文件路径
    if (!filePath.isEmpty()) {
        QElapsedTimer timer;
        timer.start(); // 开始计时
        QFile file(filePath);
        if (file.open(QIODevice::ReadOnly | QIODevice::Text)) {
            QByteArray fileData;
            QString fileContent;
            // 循环读取文件内容，直到文件末尾
            while (!file.atEnd()) {
                fileData = file.read(1024); // 每次读取 1024 字节
                // 将文件内容转换为 QString 类型，并拼接到
                fileContent 变量中
                fileContent += QString(fileData);
            }
        }
    }
}

```

```

        }

        // 将文件内容显示在 textBrowser1 控件中
        ui->textBrowser1->setPlainText(fileContent);
    }

    double elapsedTime = timer.elapsed(); // 获取经过的时间（毫秒）
    timer.invalidate(); // 停止计时器
    ui->doubleSpinBox->setValue(elapsedTime); // 将时间显示在
doubleSpinBox 中
    }
}

void file::on_pushButton2_clicked()
{
    // 弹出对话框，让用户选择保存文件的路径
    QString filePath = QFileDialog::getSaveFileName(this, tr("保存文件"), ".", tr("Text Files (*.txt)"));

    // 如果用户选择了文件，则读取文件内容并显示在文本框中
    if (!filePath.isEmpty()) {
        QElapsedTimer timer2;
        timer2.start(); // 开始计时
        QFile file(filePath);
        // 以只写和文本模式打开文件
        if (file.open(QIODevice::WriteOnly | QIODevice::Text)) {
            QString textToWrite = ui->textBrowser2->toPlainText(); //
获取文本框中的文本
            QByteArray testData(textToWrite.toUtf8()); // 将文本转换为 UTF-8 编码字节数组
            if (file.write(testData)) { // 将字节数组写入文件
                QMessageBox::information(this, tr("成功"), tr("文件已

```

```

成功保存!")));

        } else {

            QMessageBox::warning(this, tr("失败"), tr("文件保存失
败!"));

        }

    }

    double elapsedTime = timer2.elapsed(); // 获取经过的时间 (毫
秒)

    timer2.invalidate(); // 停止计时器

    ui->doubleSpinBox2->setValue(elapsedTime); // 将时间显示在
doubleSpinBox 中

    }

}

void file::on_pushButton3_clicked()
{

    // 弹出对话框, 让用户选择保存文件的路径

    QString filePath = QFileDialog::getSaveFileName(this, tr("保存文
件"), ".", tr("Text Files (*.txt)"));

    // 如果用户选择了文件, 则读取文件内容并显示在文本框中

    if (!filePath.isEmpty()) {

        QElapsedTimer timer2;

        timer2.start();

        QFile file(filePath);

        // 以追加、只写和文本模式打开文件

        if (file.open(QIODevice::WriteOnly | QIODevice::Append |
QIODevice::Text)) {

            QString textToWrite = ui->textBrowser2->toPlainText(); //
获取文本框中的文本

            QByteArray textData(textToWrite.toUtf8()); // 将文本转换

```

为字节数组

```
        if (file.write(textData)) { // 将字节数组写入文件
            QMessageBox::information(this, tr("成功"), tr("文件已
成功保存!"));
        } else {
            QMessageBox::warning(this, tr("失败"), tr("文件保存失
败!"));
        }
    }

    double elapsedTime = timer2.elapsed(); // 获取经过的时间 (毫
秒)

    timer2.invalidate(); // 停止计时器

    ui->doubleSpinBox2->setValue(elapsedTime); // 将时间显示在
doubleSpinBox 中
}

}

void file::on_pushButton4_clicked()
{
    QString filePath = QFileDialog::getOpenFileName(this, tr("打开文
件"), ".", tr("Text Files (*.txt)"));

    // 如果用户选择了文件, 则读取文件内容并显示在文本框中
    if (!filePath.isEmpty()) {
        QElapsedTimer timer2;
        timer2.start();

        QFile file(filePath);

        if (file.open(QIODevice::ReadOnly | QIODevice::Text)) {
            QByteArray textData = file.readAll(); // 读取文件中的所有
数据

            QString textToDisplay = QString::fromUtf8(textData); // 将
```

字节数组转换为 QString

ui->textBrowser1->append(textToDisplay); // 追加读取的文本到文本框中

```
    } else {
```

```
        QMessageBox::warning(this, tr("失败"), tr("文件读取失败!"));
```

```
    }
```

```
    double elapsedTime = timer2.elapsed(); // 获取经过的时间（毫秒）
```

```
    timer2.invalidate(); // 停止计时器
```

```
    ui->doubleSpinBox->setValue(elapsedTime); // 将时间显示在doubleSpinBox 中
```

```
}
```

```
}
```

图 1

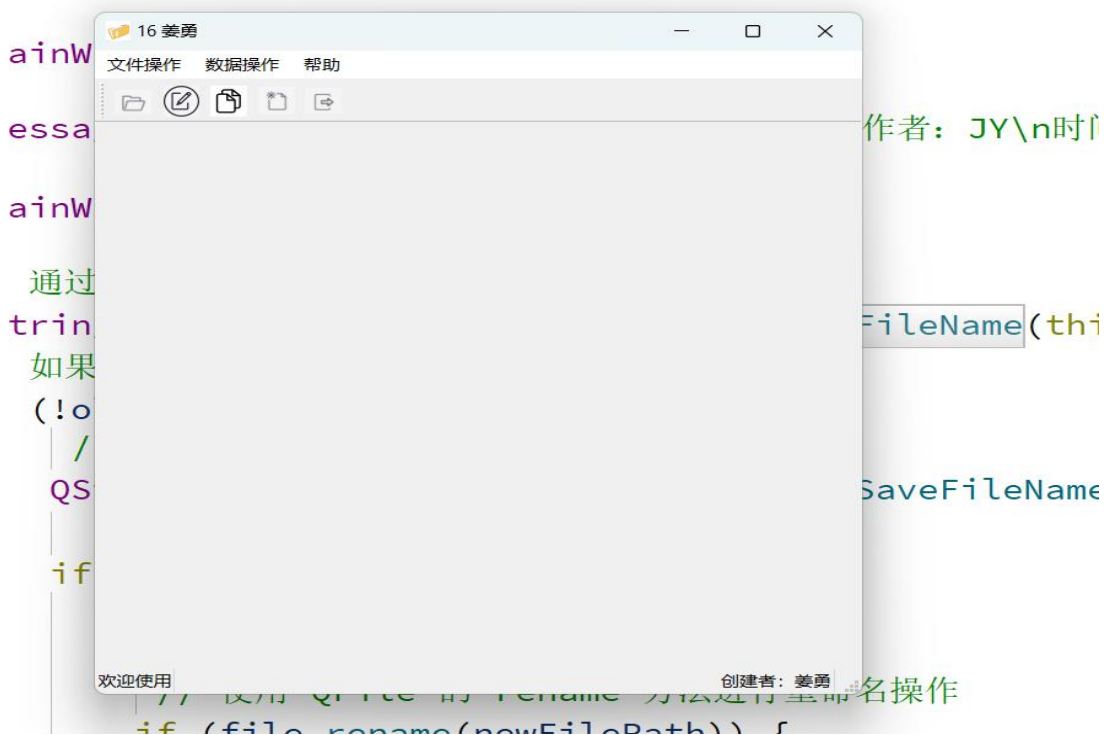


图 1 设计好的主窗口

图 2

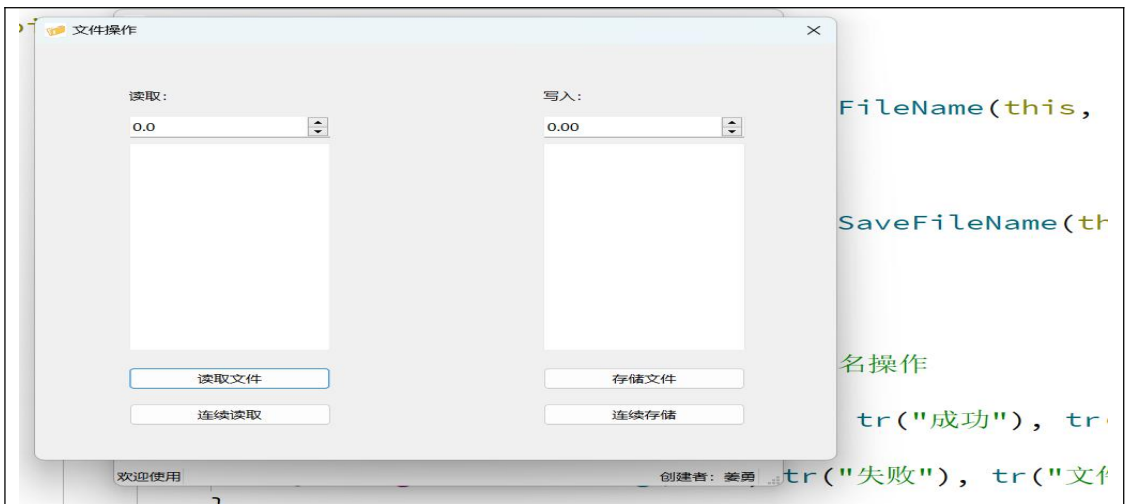


图 2 文件操作选择后，跳转到的文件读取和存储界面，有标题，执行的操作时间、内容页面和操作按钮

图 3

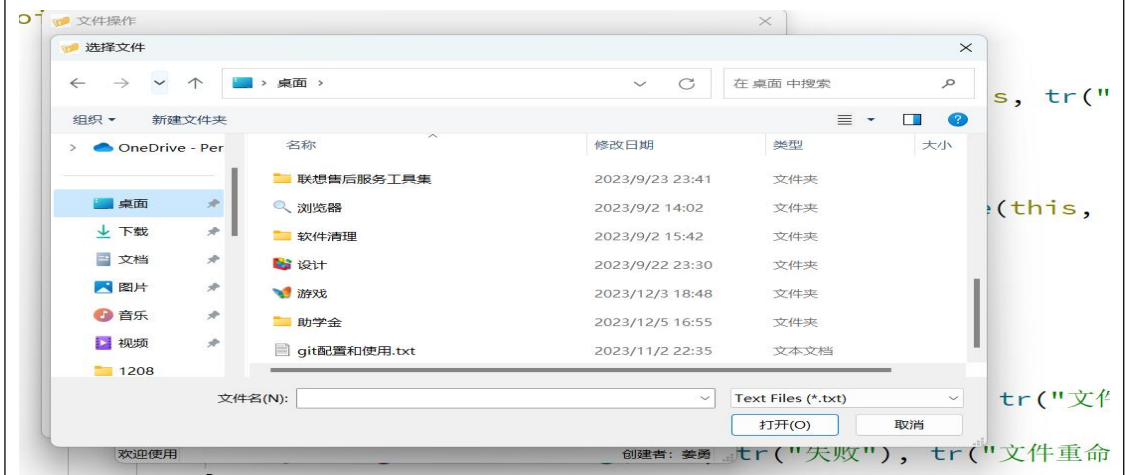


图 3 点击读取文件后弹出对话框让用户选择文件

图 4

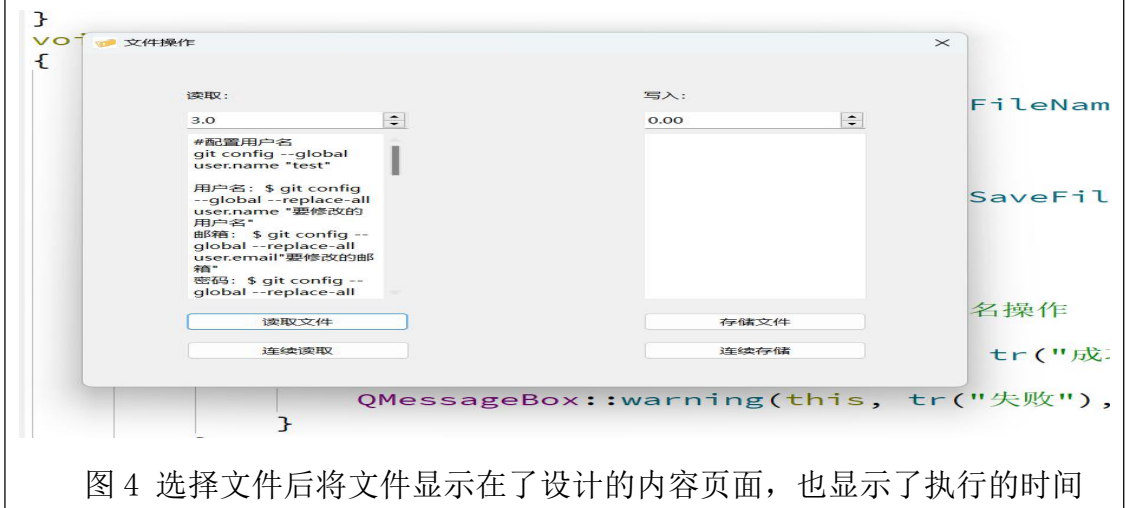


图 4 选择文件后将文件显示在了设计的内容页面，也显示了执行的时间

图 5

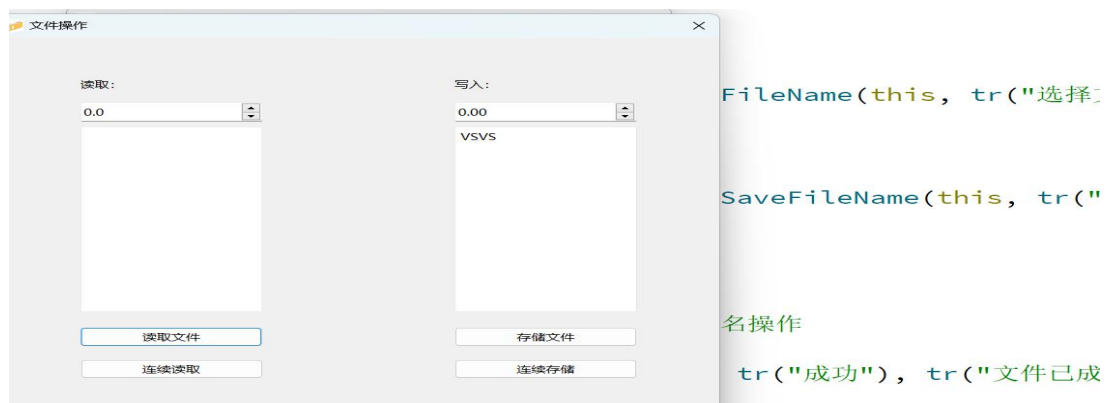


图 5 将要存储的文字输入在内容页面，然后点击存储文件操作按钮

图 6

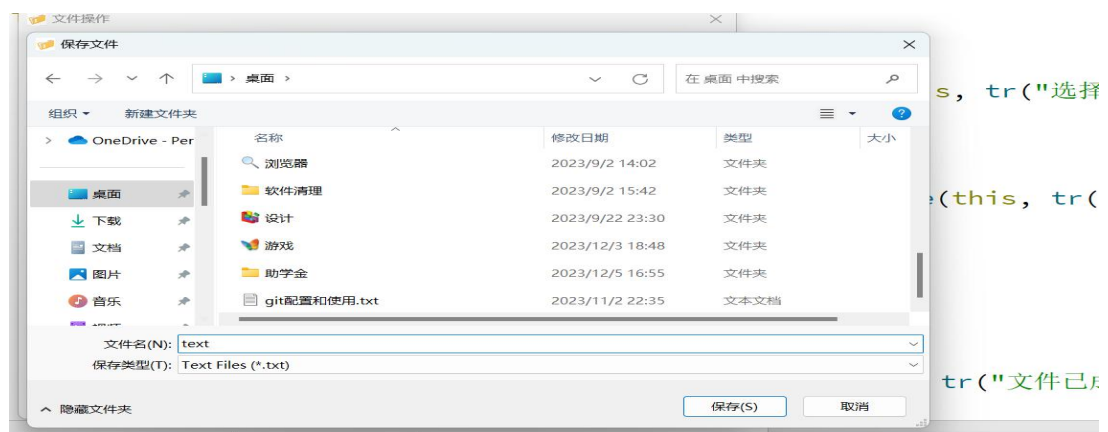


图 6 让用户选择要保存的路径和保存的文件名

图 7

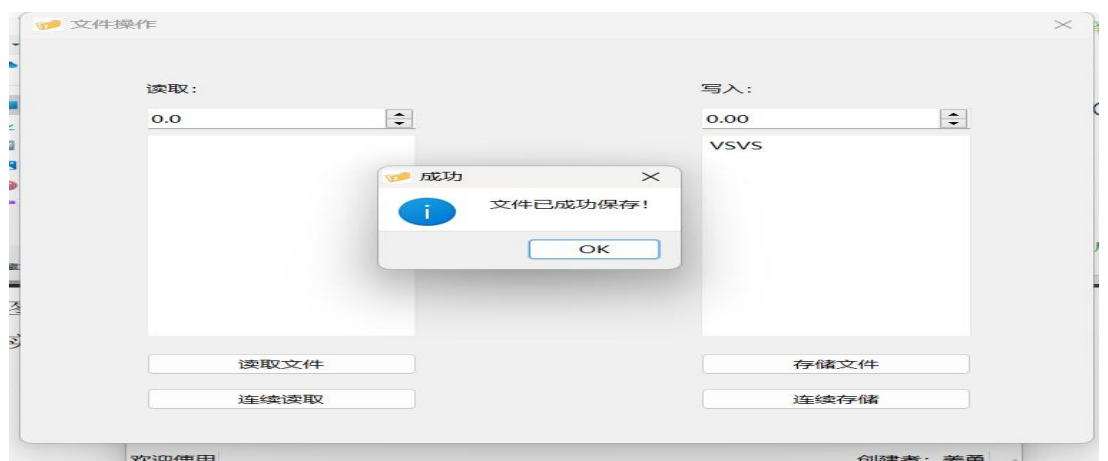


图 7 显示文件已成功保存

图 8

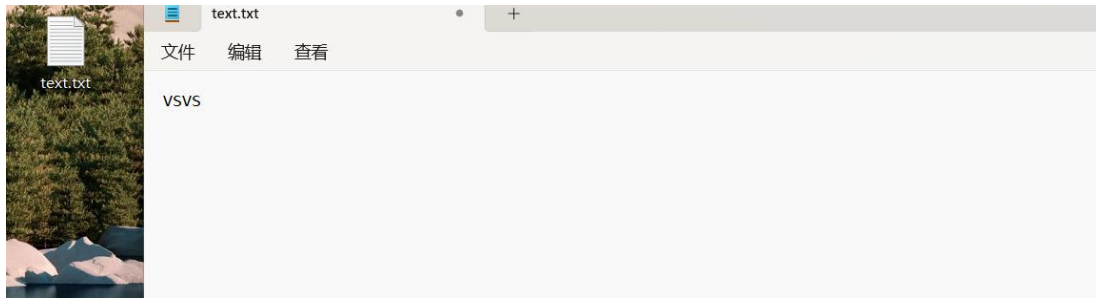


图 8 电脑已经有了保存的文件，打开后查看，要存储的内容也已经保存

图 9

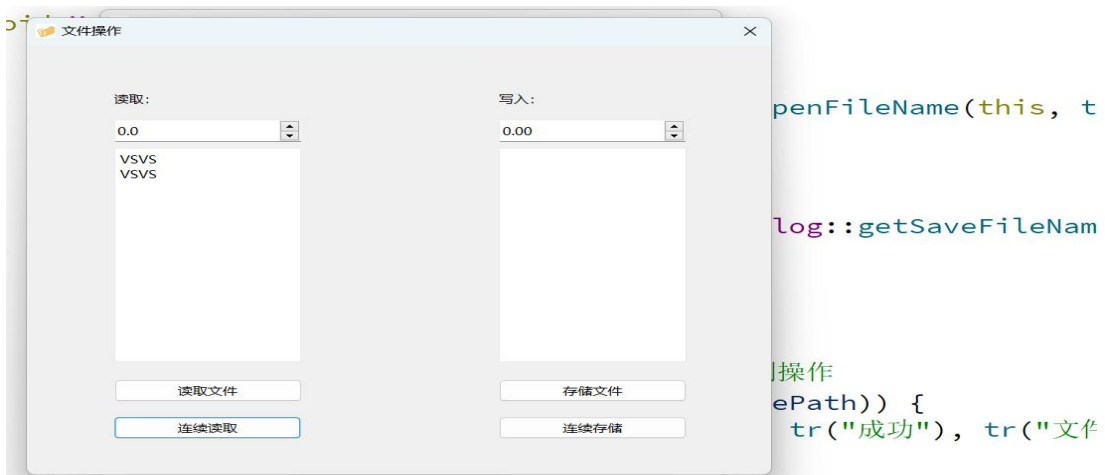


图 9 点击连续读取按钮可以将想要读取的内容追加在前面读取的内容后面

图 10

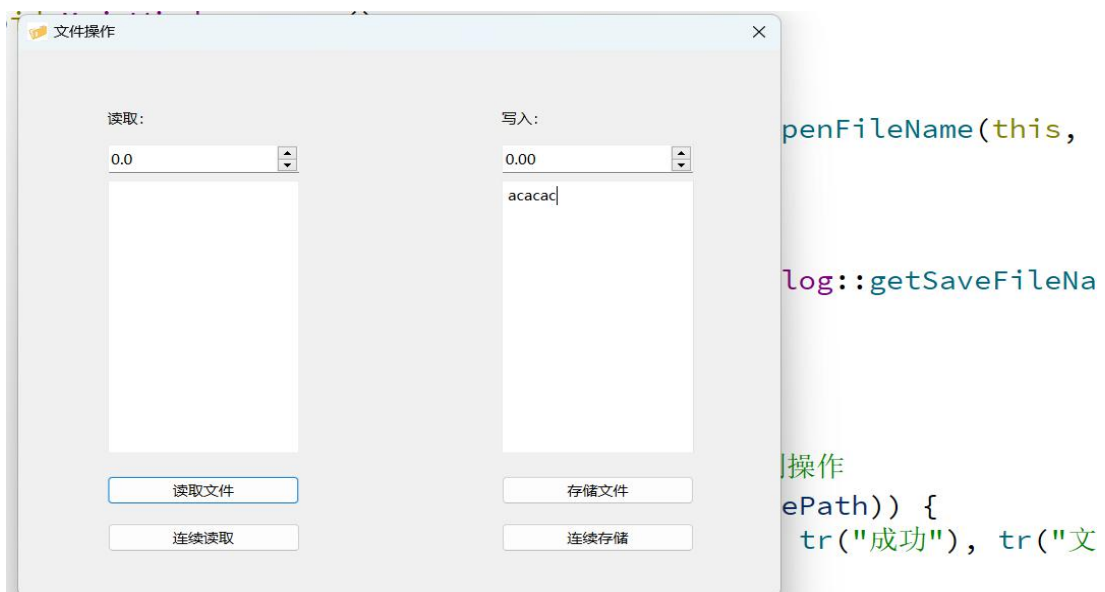


图 10 连续存储按钮也可以将向存储的内容追加到前面存储的内容后面

图 11

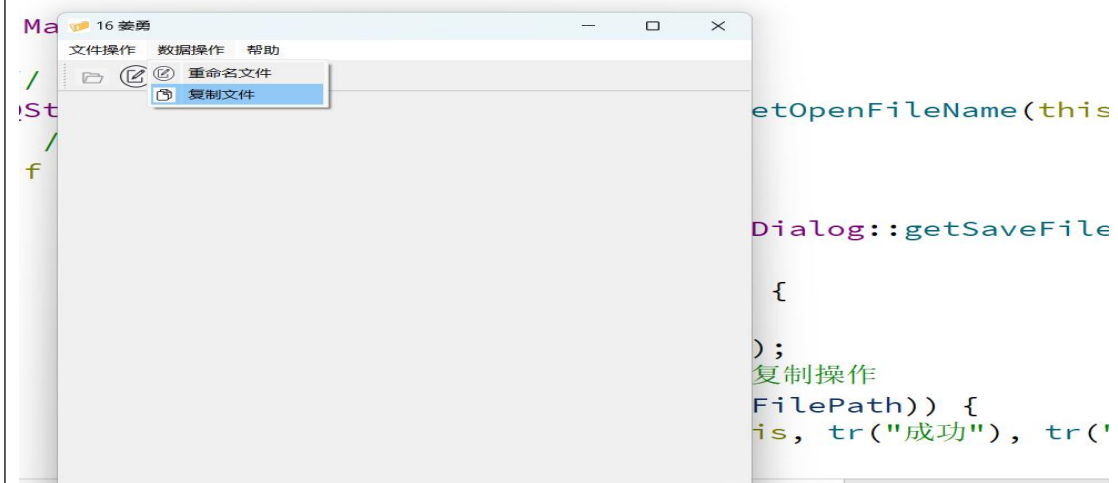


图 11 在数据操作中有重命名和复制文件功能

图 12

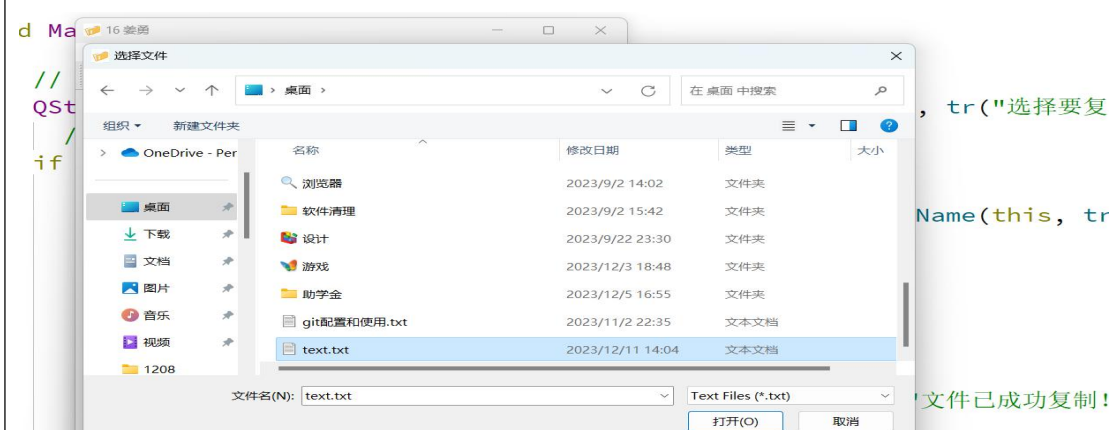


图 12 选择重命名之后，弹出对话框先让用户选择要重命名的文件

图 13

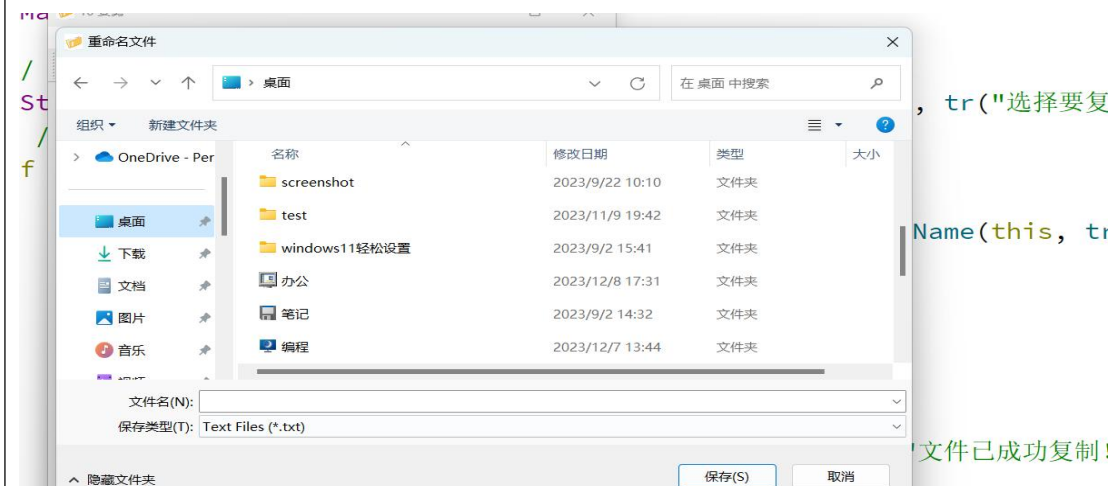


图 13 选择要重命名的文件弹出对话框选择存储重命名文件的名称和路径

图 14

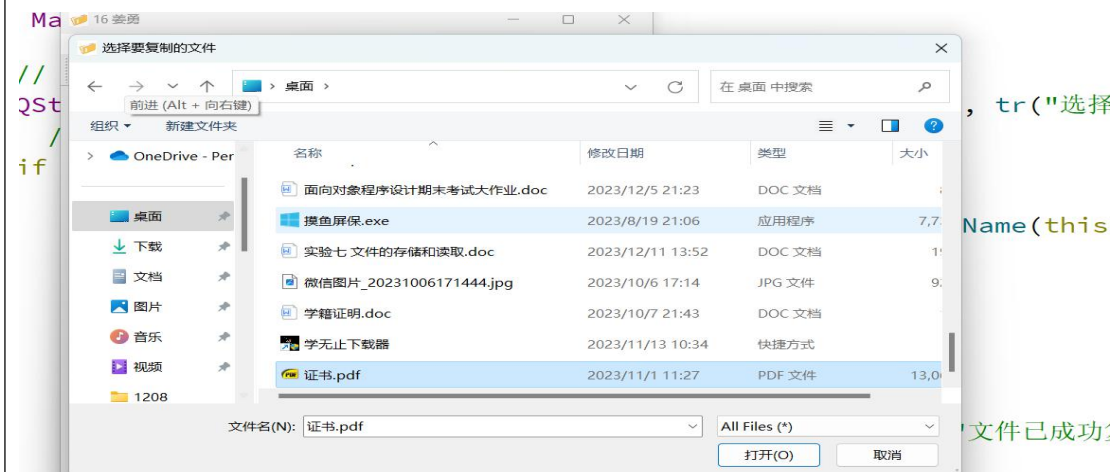


图 14 复制文件也是一样，先选择要复制的文件，选择后弹出对话框选择存储的位置和文件名

图 15

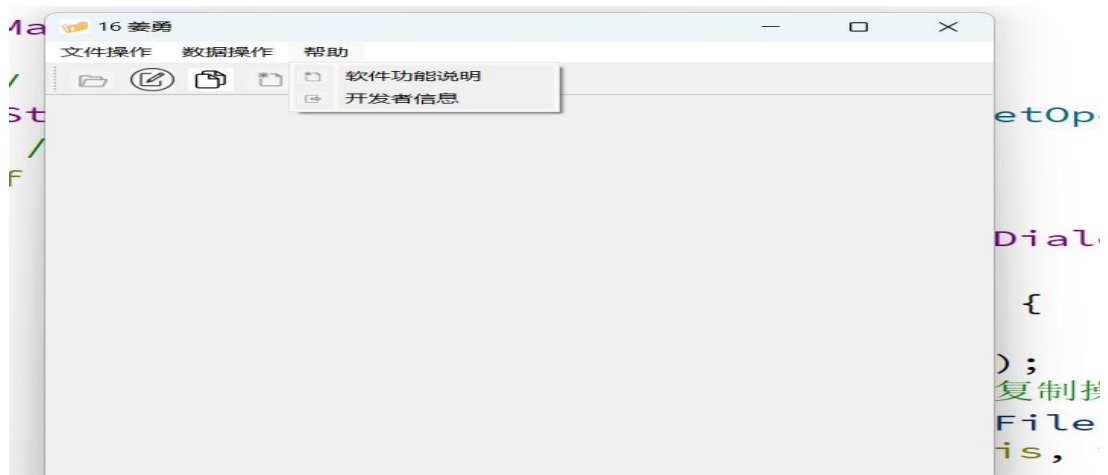


图 15 在帮助菜单中有软件的功能说明和开发者的信息

图 16

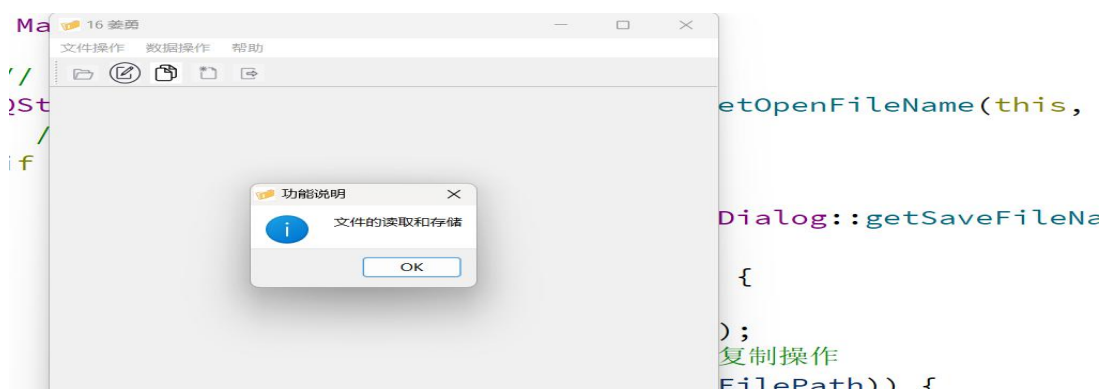


图 16 软件的功能说明

图 17

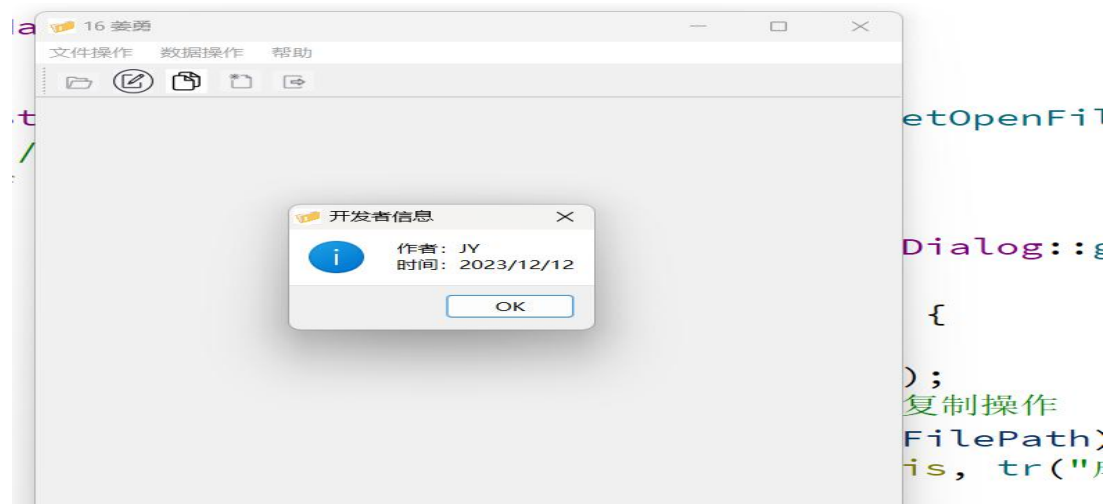


图 17 软件的开发者信息

图 18

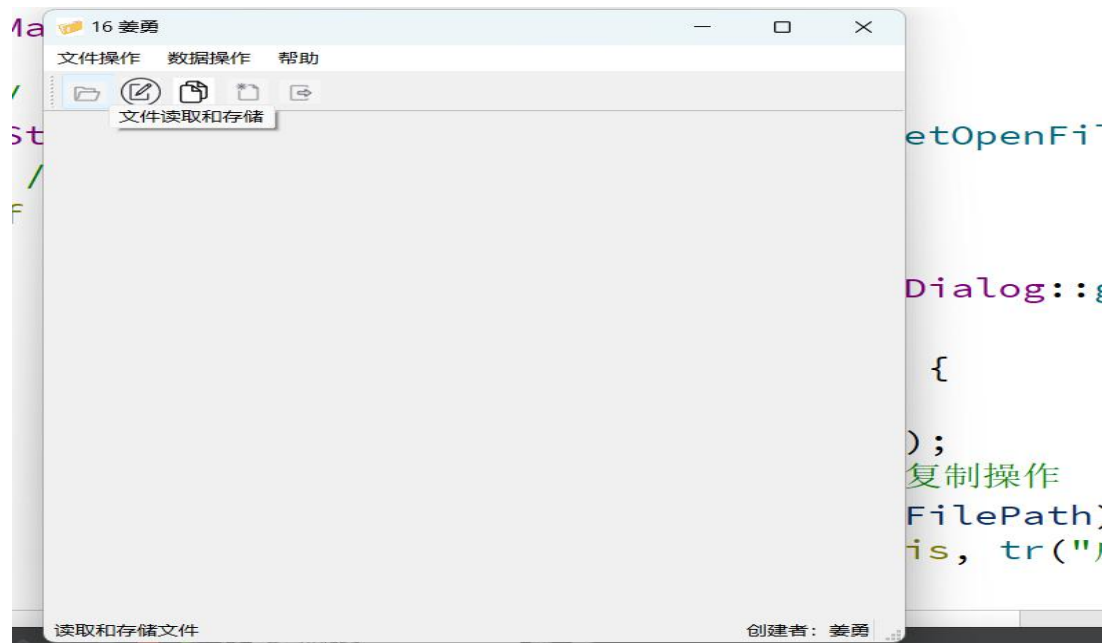


图 18 设计的工具栏和状态栏，状态栏随工具栏的选择而变化

四、实验总结

在实验中，我尝试使用 Qt 中的 QFile 来进行文件的存储和读取操作，并总结了以下实验结果和心得：

存储文件：通过创建 QFile 对象并使用 open() 函数打开文件，可以进行文件的存储操作。open() 函数的参数可以根据需要选择不同的模式，例如 QIODevice::WriteOnly 用于写入操作。使用 QTextStream 对象将数据写入到文件中，可以使用流操作符 << 来将数据写入到流中。在写入完成后，务必使

用 `close()` 函数关闭文件，以确保数据已经成功写入到文件中。

读取文件：通过创建 `QFile` 对象并使用 `open()` 函数打开文件，可以进行文件的读取操作。使用 `QIODevice::ReadOnly` 参数表示只读模式。使用 `QTextStream` 对象读取文件的内容，可以使用 `readAll()` 函数读取文件的所有内容，也可以使用其他函数逐行读取文件内容。在读取完成后，务必使用 `close()` 函数关闭文件。

文件打开状态的检查：在进行文件的存储和读取操作之前，需要使用 `open()` 函数来打开文件，并检查文件是否成功打开。可以使用 `QFile::isOpen()` 函数来检查文件是否打开成功。

实验七 成绩评定表

序号	考核项目	分值分布	成绩
1	出勤与纪律情况	10	
2	实验任务完成情况	40	
3	实验报告质量	50	
总分			
指导教师签字			